

Aula 5 — Razor Pages na Prática

Módulo: Razor Pages e UI Server-Side

Carga horária: 4 horas

Projeto base: Aula01.Web

Objetivo da aula:

Compreender o funcionamento do **Razor Pages**, entender suas diferenças em relação ao **MVC**, dominar o **PageModel**, **Handlers**, **BindProperty** e implementar um **CRUD completo**, com **validação server-side**, **layout** e **partial views**.

PARTE 1 — TEORIA (≈ 2 horas)

1 Razor Pages vs MVC

MVC	Razor Pages
Separação Controller/View	Página auto-contida
Mais arquivos	Menos boilerplate
Ideal para sistemas grandes	Ideal para CRUDs administrativos
Controller central	PageModel por página

 **Razor Pages é orientado à página**, reduzindo complexidade para telas administrativas.

2 PageModel

Cada página Razor possui:

- .cshtml → UI
- .cshtml.cs → lógica (PageModel)

Exemplo:

```
Pages/Produtos/Index.cshtml Pages/Produtos/Index.cshtml.cs
```

```
public class IndexModel : PageModel
```

3 Handlers (OnGet / OnPost)

Handlers são métodos que tratam requisições HTTP.

Handler	Uso
OnGet()	Carregar dados
OnPost()	Enviar formulário
OnPostDelete()	Ações específicas

Exemplo:

```
public void OnGet() { } public IActionResult OnPost() { }
```

4 BindProperty

Permite **binding automático** do formulário para o Model.

```
[BindProperty] public Produto Produto { get; set; }
```

- ✓ Elimina parâmetros
 - ✓ Código limpo
 - ✓ Integração com validação
-

PARTE 2 — PRÁTICA (≈ 2 horas)

Objetivo Prático

Criar um CRUD completo de Produto usando Razor Pages, com validação server-side, layout reutilizável e partial views.

1 Estrutura de Pastas Razor Pages

```
Pages |— Produtos | |— Index.cshtml | |— Index.cshtml.cs | |— Create.cshtml | |— Create.cshtml.cs | |— Edit.cshtml | |— Edit.cshtml.cs |— Shared | |— _Layout.cshtml | |— _ProdutoForm.cshtml
```

2 Página de Listagem (Index)

 Pages/Produtos/Index.cshtml.cs

```
using Aula01.Web.Data; using Aula01.Web.Models; using Microsoft.AspNetCore.Mvc.RazorPages; using Microsoft.EntityFrameworkCore;
namespace Aula01.Web.Pages.Produtos { public class IndexModel : PageModel { private readonly ApplicationDbContext _context; public
IndexModel(ApplicationDbContext context) { _context = context; } // Lista usada pela View public List<Produto> Produtos { get; set; }
```

```
public async Task OnGetAsync() { // Leitura sem tracking (melhor performance) Produtos = await _context.Produtos .Include(p => p.Categoria) .AsNoTracking() .ToListAsync(); } }
```

Pages/Produtos/Index.cshtml

```
@page @model Aula01.Web.Pages.Produtos.IndexModel <h2>Produtos</h2> <a asp-page="Create">Novo Produto</a> <table> @foreach (var p in Model.Produtos) { <tr> <td>@p.Nome</td> <td>@p.Preco.ToString("C")</td> <td>@p.Categoria?.Nome</td> <td> <a asp-page="Edit" asp-route-id="@p.Id">Editar</a> </td> </tr> } </table>
```

3 Página de Criação (Create)

Pages/Produtos/Create.cshtml.cs

```
using Aula01.Web.Data; using Aula01.Web.Models; using Microsoft.AspNetCore.Mvc; using Microsoft.AspNetCore.Mvc.RazorPages; using Microsoft.AspNetCore.Mvc.Rendering; namespace Aula01.Web.Pages.Produtos { public class CreateModel : PageModel { private readonly ApplicationDbContext _context; public CreateModel(ApplicationDbContext context) { _context = context; } [BindProperty] public Produto Produto { get; set; } public List<SelectListItem> Categorias { get; set; } public void OnGet() { Categorias = _context.Categorias .Select(c => new SelectListItem { Value = c.Id.ToString(), Text = c.Nome }) .ToList(); } public async Task<IActionResult> OnPostAsync() { if (!ModelState.IsValid) { OnGet(); // Recarrega categorias return Page(); } _context.Produtos.Add(Produto); await _context.SaveChangesAsync(); return RedirectToPage("Index"); } }
```

Pages/Produtos/Create.cshtml

```
@page @model Aula01.Web.Pages.Produtos.CreateModel <h2>Novo Produto</h2> <form method="post"> <partial name="_ProdutoForm" model="Model.Produto" /> <select asp-for="Produto.CategoriaId" asp-items="Model.Categorias"> </select> <button type="submit">Salvar</button> </form>
```

4 Partial View Reutilizável

 Pages/Shared/_ProdutoForm.cshtml

```
@model Aula01.Web.Models.Produto <label>Nome</label> <input asp-for="Nome" /> <span asp-validation-for="Nome"></span>
<label>Preço</label> <input asp-for="Preco" /> <span asp-validation-for="Preco"></span> <label> <input asp-for="Ativo" /> Ativo
</label>
```

- ✓ Reutilizável em Create e Edit
 - ✓ Código limpo
 - ✓ Padrão corporativo
-

5 Página de Edição (Edit)

 Pages/Produtos/Edit.cshtml.cs

```
using Aula01.Web.Data; using Aula01.Web.Models; using Microsoft.AspNetCore.Mvc; using Microsoft.AspNetCore.Mvc.RazorPages;
namespace Aula01.Web.Pages.Produtos { public class EditModel : PageModel { private readonly ApplicationDbContext _context; public
EditModel(ApplicationDbContext context) { _context = context; } [BindProperty] public Produto Produto { get; set; } public async
Task<IActionResult> OnGetAsync(int id) { Produto = await _context.Produtos.FindAsync(id); if (Produto == null) return NotFound();
return Page(); } public async Task<IActionResult> OnPostAsync() { if (!ModelState.IsValid) return Page();
_context.Update(Produto); await _context.SaveChangesAsync; return RedirectToPage("Index"); } } }
```

6 Layout Server-Side

 Pages/Shared/_Layout.cshtml

```
<!DOCTYPE html> <html> <head> <title>Admin .NET</title> </head> <body> <header> <h1>Sistema Administrativo</h1> <hr /> </header>
```

```
<main> @RenderBody() </main> </body> </html>
```

Validação Server-Side

- DataAnnotations no Model
- ModelState.IsValid
- Integração automática com Razor

- ✓ Segurança
 - ✓ Integridade dos dados
 - ✓ Código limpo
-

Resultado Esperado

- ✓ CRUD funcional com Razor Pages
 - ✓ Código organizado por página
 - ✓ Validação automática
 - ✓ UI server-side profissional
 - ✓ Base sólida para Blazor Server (Aula 6)
-

Atividade Avaliativa

Desafio:

1. Criar página Delete
2. Usar Partial View para confirmação
3. Implementar filtro para logar acesso às páginas